

Can This Agent Even Do That? Pre-Execution Validation for Agent Skills

Introduction

Organizations need agents that complete requested work, not just produce plausible plans. Our business goals are to **increase the delivery speed while reducing wasted model tokens**—and strengthen clients' trust by preventing agent attempts that cannot achieve the requested outcome. An authorized agent may still lack an essential tool, input, or way to coordinate its work; retries consume resources without removing that blocker. SkillC checks whether the available tools and planned steps can achieve the requested goal, and returns a verdict with evidence, before the **agent starts**. The check uses logical rules, not another model's judgment; authorization and runtime verification remain separate safeguards. We show that the check saves tokens without sacrificing completion, that savings grow when a skill is reused, that its verdicts agree with blind agent executions on 490 real skill-runtime pairs, and how it integrates with Microsoft Agent Framework and, as a runtime monitor, with the agent's own session.

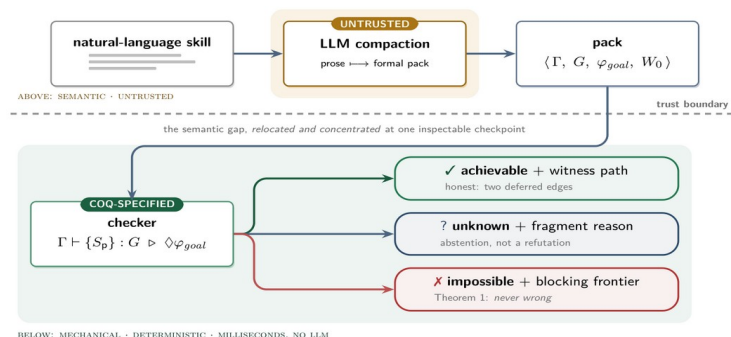


Figure 1. Check whether a task can succeed before spending model tokens on execution. Coq is a mathematical proof-checking tool used to verify the checker's rules.

Related Work

SkillC draws on classical automated planning to reason about available actions [1], later methods for checking coordination between participants [2,3,4], and logical constraint solving [5]. ADAS and AFlow generate agent workflows [6,7]; research on multi-agent failures highlights coordination problems [8]. VeriGuard addresses policy safety [9], and SkillSpector screens agent skills [10]. Translating natural language into a planning formalism, as LLM+P does before calling a classical planner [16], is itself error-prone: on Planetarium most model-written PDDL problems parse, but only about a third are semantically correct [15]. SkillC asks a complementary question—can the requested outcome be achieved with the declared tools and workflow—and treats the translation as untrusted, measuring it against executed outcomes.

Methodology

Describe the task before running it. SkillC checks a reviewed description of the goal, tools, expected effects, and starting conditions. A deterministic front end extracts it when a skill declares its tools. Otherwise a model writes the skill in SkillC Controlled English (CE), a small formal language that reads as English (Listing 1), in which each tool names the runtime tool that performs it (via) and any outside resource it needs (needs). A deterministic binder withdraws tools the runtime lacks and blocks tools whose resources are not granted, and the checker runs on the bound description. After an IMPOSSIBLE verdict the

model may repair its description once, but a deterministic guard rejects any repair that changes the goal or drops a requirement. The model never decides the verdict, and when the capabilities alone reveal an unavoidable blocker, SkillC rejects the task without any model call.

```
Skill `book-flight`.
Tool `search_fares` (owner `agent`): via `browser`; adds `flight_selected`.
Tool `book_flight` (owner `agent`): via `bash`; needs `airline_account`;
    requires `flight_selected`; adds `booked`.
Goal: `booked`.
Protocol:
- `agent` uses `search_fares`.
- `agent` uses `book_flight`.
```

Listing 1. A complete skill in SkillC Controlled English. In the developer sandbox, which grants no airline account, the binder blocks `book_flight` and the verdict is IMPOSSIBLE [BLOCKED_GUARD] naming that tool; in an office runtime with no shell or browser both tools are withdrawn and the verdict is IMPOSSIBLE [MISSING_CAPABILITY].

Explain the decision and its limits. The result is IMPOSSIBLE, ACHIEVABLE under the stated assumptions, or UNKNOWN when the check cannot decide. UNKNOWN is not approval; malformed or unsafe inputs are errors. The default check judges the declared workflow; because a rejected workflow may have a workable alternative, the goal-only policy turns a protocol error into UNKNOWN and rejects only on a capability-level refutation. Each result names the supporting steps or the blocking conditions, and what still needs runtime verification.

Data

SkillC involves no model training; all cases are test cases. **Live backend:** the same agent with and without SkillC on live tools and services, same model (GPT-5.4 through Azure OpenAI) and task conditions; 56 cases in 14 task families cover complete capabilities, missing tools and missing inputs, and completion was checked independently. **Reuse:** real model calls with simulated tools on confirmed impossible tasks. **Real skills with executed ground truth:** 490 skill-runtime pairs; the skills come from more than 20 licensed public repositories (Anthropic, Cloudflare, Expo and others) plus 20 created probes, and the runtimes are a developer sandbox, an offline workstation, and an office assistant without code execution. Every pair was executed blindly by an agent that never saw a verdict; a pair counts as impossible only when the executor's blocker was a missing runtime tool, credential, network or safety rule, and inconclusive outcomes leave the decided metrics. Compaction used one Claude model family; test sets, prompts and criteria were frozen in commits before any compaction ran; a direct model verdict given the same runtime description is the baseline. We measured tokens, completion, incorrect rejection and admission, and inconclusive decisions; the supplement [11] gives model versions, prompts and detailed results.

Results

Live execution: 47% fewer tokens. Checking tasks before execution reduced total model-token use by 46.74% (18,758 vs. 35,220) while both configurations completed all 28 achievable tasks. With the task's goal and tools written down in advance (contract-bound), SkillC blocked all 28 impossible tasks with no false positive, and token use fell in every task family (28.2%–58.9%). A model is used only when the deterministic front end cannot compile the skill, once per skill and runtime rather than per request, and its output must pass the deterministic parser, binder and checker, which alone decide the verdict.

Repeated use: 90% fewer tokens. When the same impossible task is requested again, SkillC can reuse its prepared task description rather than pay that preparation cost each time. This reduced token use by 90.47% across repeated requests in the separate simulated-tool study. This number excludes a task whose goal was actually achievable but rejected by SkillC, because preventing a possible success is not a saving.

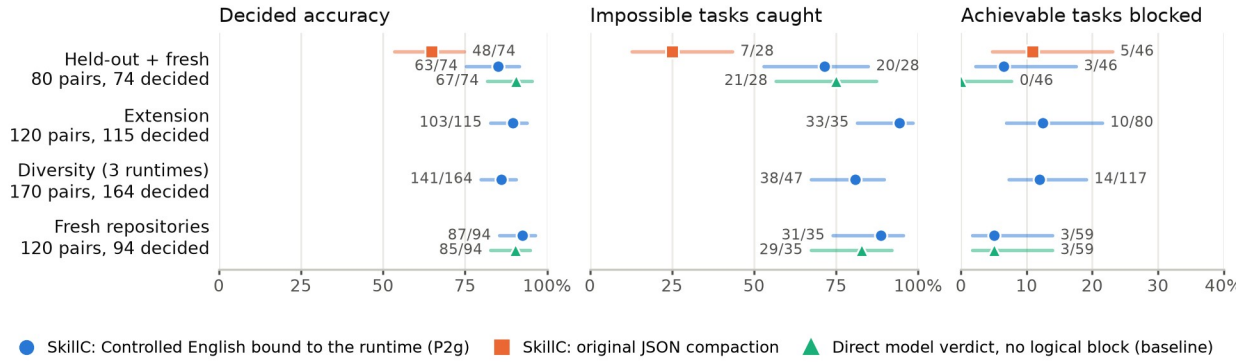


Figure 3. Verdicts on real skills against blind-execution ground truth (95% Wilson intervals). Each row is a test set frozen before compaction; the JSON and direct-verdict arms ran only where shown. Labels: correct/decided, caught/impossible, blocked/achievable.

Real skills: 88% of verdicts agree with execution. With only the skill's own instructions and no contract, runtime-bound CE compaction (P2g) decided 447 of the 490 executed pairs. Over the five test sets (pooled after the fact; Figure 3 merges the two smallest), 394 verdicts (88%) agreed with the executed outcome; P2g caught 122 of 145 impossible pairs (84%) and wrongly blocked 30 of 302 achievable ones (10%). Accuracy held on 100 non-developer skills from 21 domains and on two runtimes the method was not designed on. Against the original JSON compaction on the same 74 decided pairs, P2g was right where JSON was wrong 18 times and the reverse 3 times (exact McNemar $p = 0.0015$); most JSON errors were ordinary tool requirements left undeclared. A direct model verdict given the same runtime description matched P2g's accuracy (0.905 vs. 0.893 on 168 pairs, $p = 0.82$) at 40% of the tokens. The logical block therefore does not buy accuracy. It buys a checkable witness (which tool, which missing resource), a verdict that can be re-checked without a model call when the runtime or a policy changes. Two attempted improvements failed their pre-registered fresh tests and were dropped; the remaining errors are mostly optional external steps, such as publishing, that the compactor treated as core.

More than tool checks: impossible coordination. SkillC also checks guards, numeric constraints and protocol realizability. Example: a main agent privately picks a security or performance review, and an editor must publish the matching report; every tool exists, but the editor is never told the choice, so SkillC returns IMPOSSIBLE [NON_PROJECTABLE] before any agent launches. The fix is a protocol message (a branch label), not another tool.

Integration: gate, session hook and runtime monitor

SkillC is integrated in a prototype fork through Microsoft Agent Framework AgentMiddleware [12,13]: SkillcHook implements MAF's `process(context, call_next)` interception pattern. Before model invocation it selects the applicable SKILL.md, compiles it against a reviewed capability profile, and runs the deterministic checker. For an IMPOSSIBLE verdict it assigns the explanation to `context.result` and returns without calling `call_next()`, so neither model nor tools run; for ACHIEVABLE and, by prototype policy, UNKNOWN it awaits `call_next()` and continues through the normal MAF pipeline. The verdict, reason, blocking frontier and pack digest are retained in run metadata and OpenTelemetry traces; MAF DevUI inspects this behavior [14].

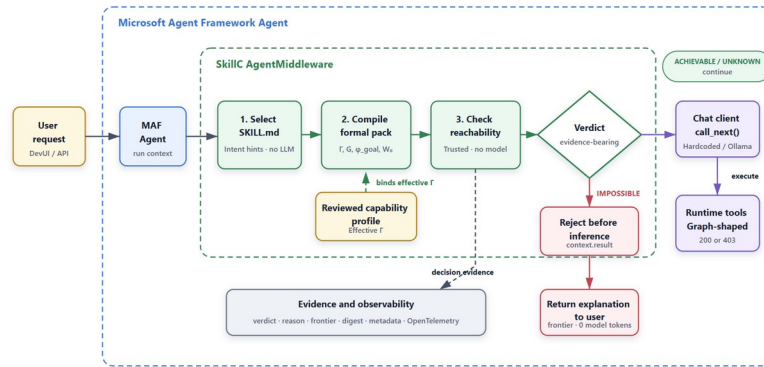


Figure 4. Implemented Microsoft Agent Framework integration, simplified from the supplied architecture. Impossible requests stop before model use; the prototype makes inconclusive requests continue.

From gate to session hook and monitor. The same checker also runs inside the agent's own session. A pre-session hook audits each proposed skill against the tools the host will grant, read from an MCP server's tools/list where available, and excludes impossible ones before the model starts. A runtime monitor, installed as Claude Code hooks, requires the agent to write its plan in CE and approves implementation only when the plan checks ACHIEVABLE; it re-checks the plan against observed failures (a missing program, a rejected credential) and revokes it once impossible. In four live scenarios with a headless coding agent, the monitor admitted an achievable fix, kept an uncredentialed deploy and a forbidden push from happening, and stopped a run once a required program proved missing (four scenarios, not a measured false-stop rate).

Keep the safeguards separate. Record verdicts for review, recheck whenever the goal or capability profile changes, retain independent authorization and runtime verification, and define a policy for inconclusive results before wider deployment: UNKNOWN is an abstention, not evidence of success.

Why the check is credible. The formal analysis states when an IMPOSSIBLE result rules out success for the task as described. Reachability refutations (a missing tool, a guard that never holds, a goal atom no tool establishes) are covered by a machine-checked abstraction theorem; protocol refutations (an unobserved choice, a non-conforming role) refute the declared workflow, not every alternative, and are reported as such. Selected results are machine-checked, the whole application is not; the supplement [11] gives the proofs and which result covers each verdict.

Implications / Conclusion

SkillC's value is simple: identify work that cannot succeed before wasting any agent token, and keep the same checker active while the agent works. Start with reviewed descriptions, use model help only when needed, and measure savings alongside completion. **Future work:** a benchmark for the runtime monitor (tokens saved on impossible runs, achievable runs stopped in error), the core-versus-optional distinction behind most remaining compaction errors, and severity-aware protocol typing that forbids a wrong branch choice only when it would be catastrophic [11].

References

- [1] R. Fikes and N. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artificial Intelligence* 2(3-4), 1971.
- [2] K. Honda, N. Yoshida, and M. Carbone. Multiparty Asynchronous Session Types. *Journal of the ACM* 63(1), 2016.
- [3] F. Barbanera, M. Dezani-Ciancaglini, and U. de'Liguoro. Un-projectable Global Types for Multiparty Sessions. *PPDP*, pp. 15:1-15:13. ACM, 2024.
- [4] F. Dagnino, P. Giannini, and M. Dezani-Ciancaglini. Deconfined Global Types for Asynchronous Sessions. *Logical Methods in Computer Science* 19(1), pp. 1-41, 2023.
- [5] L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. *TACAS, LNCS 4963*, pp. 337-340. Springer, 2008.
- [6] S. Hu, C. Lu, and J. Clune. Automated Design of Agentic Systems. *ICLR*, 2025.
- [7] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. AFlow: Automating Agentic Workflow Generation. *ICLR*, 2025.
- [8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why Do Multi-Agent LLM Systems Fail? *arXiv:2503.13657*, 2025.
- [9] L. Miculicich, M. Parmar, H. Palangi, K. D. Dvijotham, M. Montanari, T. Pfister, and L. T. Le. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. *arXiv:2510.05156*, 2025.
- [10] N. Paz, K. Pradeep, N. Raghavan, A. Nikirk, and M. Gupta. SkillSpector: A Pre-Publication Security Control for Agent Skills. *Agent Skills Workshop at CAIS*, 2026.
- [11] SkillC MLADS supplement: implementation details, model versions, prompts, evaluation setup, and detailed results. *Supplementary material (MLADS_supplementary.pdf)*, 2026.
- [12] Microsoft. "Agent Middleware." Microsoft Agent Framework documentation. <https://learn.microsoft.com/en-us/agent-framework/concepts/agents/middleware/>
- [13] Microsoft. "Python Agent Middleware Samples." Microsoft Agent Framework, GitHub. <https://github.com/microsoft/agent-framework/tree/main/python/samples/02-agents/middleware>
- [14] Microsoft. "Agent Framework DevUI." Microsoft Agent Framework, GitHub. <https://github.com/microsoft/agent-framework/blob/main/python/packages/devui/README.md>
- [15] M. Zuo, F. Piedrahita Velez, X. Li, M. L. Littman, and S. H. Bach. Planetarium: A Rigorous Benchmark for Translating Text to Structured Planning Languages. *NAACL*, 2025. *arXiv:2407.03321*.
- [16] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. *arXiv:2304.11477*, 2023.